Лабораторная работа №2
«Векторное время»

Введение

При решении задачи подсчета полной суммы денег с помощью скалярных часов предполагалось, что всем процессам известен некоторый момент логического времени *t*, в который процессы будут подсчитывать денежные средства на своих счетах и в каналах между филиалами. Однако сама процедура достижения соглашения между процессами о таком будущем моменте *t* представляет определенные сложности. Действительно, при использовании механизма скалярных часов процесс не может произвольным образом выбрать значение *t* для согласования с другими процессами, т.к. не исключено, что на момент выбора *t* логические часы других процессов ушли далеко вперед, и их текущее логическое время потенциально может превышать любое предлагаемое значение *t*. Кроме того, если *t* окажется слишком большим, то придется долго ожидать наступления этого момента. Использование векторных часов может помочь в решении этой задачи.

Исходные данные

- Число N процессов, составляющих распределенную банковскую систему;
- Начальные балансы S для каждого из счетов (один процесс — один счет),

где N и S — целые числа. При автоматической проверке $N \in [2; 10]$, $S \in [1; 99]$.

Постановка задачи

За основу банковской системы необходимо взять модель распределенной системы, описанную в условии к подготовительной работе. Разделяют два типа процессов, составляющих банковскую систему:

- процессы, принимающие запросы от клиентов (тип «К»);
- процессы, отвечающие за обслуживание счетов (тип «С»).

В данном этапе лабораторной работе используется один процесс типа «К» и $N - 1$ процессов типа «С». Каждый процесс имеет собственные логические часы. Процессы обмениваются сообщениями в асинхронном режиме.

Процесс «К» на основе запросов клиента формирует запросы к соответствующим процессам «С». Поддерживаются следующие операции:

- перевод денег между счетами;
- получение информации об истории изменения баланса.

При переводе денег процесс «К» отправляет сообщение процессу «C_{src}», с которого осуществляется списание. Процесс «$C_{src»}$ после выполнения требуемых операций пересылает исходное сообщение процессу «C_{dst}». Процесс «C_{dst}» после выполнения всех необходимых действий отправляет процессу «К» подтверждение о выполнении операции. Предполагается, что используются корректные запросы на перевод и каналы надежные, поэтому другие подтверждения о выполнении операций не требуются.

Задание

Используя топологию распределенной системы и библиотеку межпроцессного взаимодействия (IPC) из подготовительной работы, необходимо реализовать банковскую систему, описанную выше.

Во время выполнения программы осуществляются переводы денег между счетами. При завершении на экран выводится таблица с информацией о балансе каждого счета и полной сумме денег, находящихся на всех счетах, в каждый момент времени $t \in 0, 1, \ldots, T$. Последняя отметка времени T определяется на момент завершения каждого из процессов «С» (при получении соответствующего сообщения от процесса «К», см. далее), $T \leq MAX_T$.

При запуске программы, как и раньше, указывается число дочерних процессов в полносвязной топологии. Последние X параметров задают начальные балансы для каждого из счетов. Например, следующая команда:

./pa7 -p 3 10 20 30

означает, что в банковской системе три счета (точнее три филиала, каждый из которых обслуживает всего один счет) с идентификаторами 1, 2, 3 и с начальными балансами $10, $20, $30 соответственно.

В реализации банковской системы необходимо использовать векторное время. Для этого при отправке любого из сообщений процесс должен передавать свое текущее векторное время, а при получении сообщения обрабатывать показания векторных часов процесса-отправителя.

Векторные часы представляют собой массив элементов типа *local_time_t.* Размер вектора равен числу процессов, составляющих распределенную систему. Показания векторных часов добавляются в сообщение при вызове функций *send**() и обрабатываются при получении сообщений в функциях *receive*()*. В канал массив должен передаваться после заголовка сообщения *MessageHeader* и перед телом сообщения *payload*. При этом длина сообщения *s_len* должна быть модифицирована соответствующим образом. Стоит отметить, что это действие абсолютно прозрачно для пользователей данных функций. Поле заголовка *s_local_time* перенесено в *s_reserved[]*. Доступ процесса к своим векторным часам и тип памяти для их хранения не специфицированы.

Процесс «К» реализуется на основе родительского процесса, процессы «С» — на основе дочерних процессов подготовительной работы. После того, как получены сообщения *STARTED* от всех процессов «С», процесс «К» должен вызвать функцию *bank_robbery()*, которая выполняет ряд переводов денег между произвольными процессами «С» посредством вызовов функции *transfer()*. Перевод описывает структура *TransferOrder*, передаваемая сообщением типа *TRANSFER*. При выполнении перевода процесс «К» отправляет сообщение *TRANSFER* процессу «C_{src}», после чего переходит в режим ожидания подтверждения (пустое сообщение типа *ACK*) процессом «C_{dst}» получения перевода. Переводы могут быть инициированы только процессом «К».

После выполнения функции *bank_robbery()* процесс «К» отправляет сообщение *STOP* всем процессам «С» и дожидается получения сообщения DONE от всех дочерних процессов.

«Полезной» работой процесса «С» является ожидание и обработка сообщений двух типов: *TRANSFER* и *STOP*. При получении сообщения *TRANSFER* процесс «C_{src}» после выполнения всех необходимых операций пересылает это сообщение процессу «C_{dst}». Процесс «C_{dst}» обрабатывает сообщение и отправляет сообщение ACK процессу «К».

При получении сообщения *STOP* процесс «С» переходит к выполнению третьей фазы. Во время выполнения третьей фазы процесс «С» может получать сообщения «*TRANSFER*» от других процессов «С», при этом гарантируется, что после отправки сообщения *STOP* процесс «К» не выполняет новых переводов.

Для подсчета полной суммы денег добавлены новые типы сообщений: *EMPTY, SNAPSHOT_VTIME, SNAPSHOT_ACK.* Сообщение *SNAPSHOT_VTIME* используется для согласования векторного времени *t*, когда необходимо выполнить подсчет денег, и содержит время следующего, еще не наступившего события процесса-отправителя *P*. Процесс, получивший сообщение *SNAPSHOT_VTIME,* должен отправить в ответ пустое сообщение типа *SNAPSHOT_ACK.* Отправка и получение *SNAPSHOT_VTIME* и

SNAPSHOT_ACK не должны влиять на показания векторных часов процессов. Сообщение *EMPTY* можно использовать для продвижения времени процесса-получателя. Каждый процесс, получивший *SNAPSHOT_VTIME,* должен передать процессу *P*, инициировавшему подсчет денег, сообщение типа *BALANCE_STATE* после того, как наступил момент времени *t* и известно состояние каналов. При получении информации, необходимой для подсчета полной суммы денег, процесс *P* выводит сумму на экран в виде «*[t_1, t_2, … , t_n] $balance $pending\n*», где в квадратных скобках указывается время *t.*

Процедура согласования векторного времени *t* для подсчета полной суммы денег инициируется родительским процессом посредством вызова функции *total_sum_snapshot()*. Данная функция вызывается родительским процессом из функции *bank_robery()* и должна быть реализована студентами на основе алгоритма, представленного на лекции.

При реализации векторных часов считать, что у процессов отсутствуют внутренние события. События отправки и получения сообщений *SNAPSHOT_VTIME* и *SNAPSHOT_ACK* не участвуют в продвижении векторного времени. При наступлении каждого события показания логических локальных часов процесса увеличиваются на единицу.

В дополнение к событиям из предыдущего этапа (строки форматирования изменены) для процессов типа «C» добавлено два новых события:

• процесс отправил перевод на другой счет (*log_transfer_out_fmt*);

• процесс получил перевод с другого счета (*log_transfer_in_fmt*).

Для реализации асинхронного обмена сообщениями необходимо использовать неблокирующие функции *read()* и *write()*. Как и в предыдущем этапе, в реализации запрещается использовать многопоточность: один процесс — один поток. Кроме того, нельзя использовать разделяемую память, примитивы синхронизации (семафоры и т.п.), функции *select()* и *poll()*. Логирование (*events.log*, *pipes.log* и терминал), как и в предыдущем задании.

Требования к реализации и среда выполнения

Реализацию необходимо выполнить на языке программирования Си с использованием предоставленных заголовочных файлов и библиотеки из архива *pavt_starter_code.tar.gz*.

Работа после защиты присылается в виде архива с именем *pa7.tar.gz*, содержащим каталог *pa7.* **Автоматическая проверка лабораторной работы ботом не поддерживается.**

Среда выполнения — Linux (Ubuntu версии ≥ 16.04). При проверке используется следующая команда: *gcc -std=c99 -Wall -pedantic -L. *.c*. При наличии варнингов работа не принимается. При успешном выполнении запущенные процессы не должны использовать *stderr*, код завершения программы должен быть равен 0.